

D1: Lean Introduction

From formal proofs to basic tactics

AI4Math Summer School

July 15, Tuesday

Learning Goals

After this session, you should be able to:

- explain why Lean can check proofs (small kernel, type theory);
- read type judgments: $a : A$ means “a has type A”;
- write simple functions using `def` and inspect them with `#check`, `#eval`, `#reduce`;
- understand the Curry–Howard correspondence: propositions as types, proofs as terms;
- use the core tactics: `intro`, `exact`, `apply`, `constructor`, `cases`, `contradiction`;
- use `rfl`, `rw`, and `simp` for equational reasoning;
- work with AI tools with appropriate skepticism.

Four-Part Outline

Part I	Formal Proofs and AI History of proof checking, Lean and Mathlib, AI tools
Part II	Types and Functions Type judgments, #check, functions, implicit arguments
Part III	Curry–Howard and Prop Propositions as types, logical connectives as constructions
Part IV	Logic Tactics intro, exact, apply, rfl, rw, simp

Six-Day Course Overview

Day	Date	Topic
D1	Jul 15 (Tue)	Types, Prop, basic tactics, AI tools
D2	Jul 16 (Wed)	Inductive types, structures, type classes
D3	Jul 17 (Thu)	Instances, AI workflows, algebra, SimpleGraph
D4	Jul 18 (Fri)	Analysis, topology, filters, limits
D5	Jul 19 (Sat)	Functional programming, Monad
D6	Jul 20 (Sun)	Algorithm verification, termination, complexity

Tutorial and Project Schedule

Afternoon tutorials

- D1: environment setup; AI tool configuration; Mathlib search; `ring/linarith/decide`
- D2: inductive, structure, class exercises
- D3: algebra/combinatorics exercises; project selection and grouping
- D4–D5: project work; AI drafting, Mathlib search, Lean repair
- D6: common feedback on spec, invariants, termination, complexity

Project timeline

- D3: form groups, select topics
- Weekend D4–D5: independent group work with AI tools
- D6: whole-class feedback and wrap-up

- Why does Lean know whether a proof is correct?
- Where does Lean fit in the history of mathematics?
- How should we use AI tools alongside Lean?

Search, Proof, and Verification Working Together

In 2023–2025, a Knuth problem was resolved by a combination of:

- AI search — finding candidate constructions;
- human proof — organizing concepts and proof routes;
- Lean formalization — producing a checkable artifact.

This is not a story of “AI does mathematics alone.” It is a story of search, explanation, computation, and proof-checking working together.

From Recording to Checkable Proofs

Mathematics has passed through four stages:

- 1 **Recording** — quantities, accounts, calendars.
- 2 **Computation** — obtaining results by rule.
- 3 **Proof** — explaining *why* something must hold.
- 4 **Formalization** — making the checking rules mechanical.

Lean operates at stage 4: a proof written in Lean is an object that the kernel checks step by step against explicit rules.

Ancient Mathematics: Recording and Computing

Early mathematical traditions (Babylonian, Egyptian, Greek) primarily served memory: quantities, land, debt, astronomy.

- Structured computation appears before modern proof conventions.
- The Greek tradition pushed toward “why must this be true” beyond individual examples.
- Formalization inherits a long requirement: records must be accurate, computations must be reliable, reasons must be inspectable.

Euclid: Proofs as Dependency Networks

Elements (c. 300 BCE) established a model still in use:

- Start from definitions, postulates, and axioms.
- Order propositions by their dependencies.
- Diagrams aid intuition; proofs carry the conclusions.

The lasting contribution is not only the geometry but the idea of arranging mathematical knowledge into a *traceable dependency chain*. Lean's dependency graph is a direct descendant.

Al-Khwarizmi: Algorithms and Knowledge Preservation

Al-Khwarizmi (c. 780–850) contributed two lasting ideas:

- **algebra** — transforming problems into symbols and rules;
- **algorithm** — steps that can be executed repeatedly.

Mathlib inherits not a single theorem but the tradition of *organizing steps and knowledge for reuse*.

Modern Proofs Grow Long: Fermat to Wiles

Fermat's Last Theorem:

$a^n + b^n = c^n$, $n > 2$ has no positive integer solution.

- Short to state; centuries to prove.
- Wiles (1995): semi-stable case via modular forms and Galois representations.
- Complete modularity theorem completed by subsequent work.

As proofs grow longer, the question of *who checked each step* becomes pressing.

When Verification Becomes Part of the Story

- 2012: Mochizuki's IUT papers claim a proof of the ABC conjecture.
- 2018: Scholze and Stix identify a gap in a key step.
- Formalization projects try to turn the dispute into inspectable definitions and dependencies.

The mathematical community needs not only answers but *a form of expression that can be jointly inspected*.

Hilbert: Can Proof Checking Be Mechanized?

David Hilbert (1862–1943) aimed to:

- place mathematics on an explicit formal foundation;
- express proofs in a clear symbolic language with fixed rules;
- prove the consistency of those foundations.

Machine-checked proofs are a natural endpoint of this program: instead of trusting authority or intuition, every step is held to explicit rules.

Principia Mathematica: Reasoning as Symbols

Russell and Whitehead (1910–1913) aimed to derive mathematics from explicit symbolic rules.

- Famous: $1 + 1 = 2$ required hundreds of pages.
- Demonstrated the power of formalization — and its writing cost.
- The cost motivated the search for automation.

Gödel: Limits Are Boundaries, Not Endpoints

Gödel's incompleteness theorems (1931):

- A sufficiently strong consistent formal system cannot prove all true statements.
- Such a system typically cannot prove its own consistency internally.

This does not prevent us from mechanically checking concrete proofs within a fixed system. Lean is not a machine that proves all truths. It is a tool that checks a given formal proof.

Two Routes: Proof Search vs. Proof Checking

Proof Search

Machine tries to find a derivation.

- 1956: Logic Theorist
- Today: ATP solvers, AI provers

In Lean, the two converge: tactics and AI help with search, but every proof must pass the kernel.

Proof Checking

The proof object is supplied; kernel verifies it.

- 1968: Automath
- Today: Coq, Lean kernel

Nicolaas de Bruijn's Automath (1968):

- First system to express proofs as precisely typed objects.
- Every line is checked against a rule.
- A small trusted kernel suffices.

The key shift: from “reading a text” to “checking an object.” This is exactly what the Lean kernel does.

Four-Color Theorem: Computer-Assisted Proof

- 1976: Appel and Haken prove the four-color theorem using computer case analysis.
- Question: can the mathematical community trust computer output?
- 2005: Georges Gonthier's team completes a machine-checked proof in Coq.

This binds two questions together: “Is the computation correct?” and “Is the proof checkable?”

Flyspeck: Engineering a Complex Proof

- 1611: Kepler's sphere-packing conjecture.
- 1998: Thomas Hales announces a computer-assisted proof.
- 2003–2014: Flyspeck project formalizes it in HOL Light and Isabelle.

A modern proof is often a project: the target theorem is at the top; beneath it is a large body of reusable infrastructure. Formalization is not a one-time translation; it is long-term library construction.

Proof Assistants as Infrastructure

Year	Development
1968	Automath — first proof objects
1970s	Mizar, LCF — formal methods
1980s	HOL, Coq, Isabelle — logic + automation
1994	QED Manifesto — call for a checked corpus
2013+	Lean — kernel + tactics
2017+	Mathlib — community library

The Trust Chain of Formal Proofs

- ① **Mathematical intent** — the natural-language goal.
- ② **Formal statement** — the theorem in Lean.
- ③ **Proof script** — `by ... tactic ...`
- ④ **Kernel check** — type-checking the proof term.
- ⑤ **Accepted** — valid in the specified formal system.

Lean checks whether the formalized statement has a valid proof. Mathematicians must separately verify that the statement captures the intended mathematics.

Lean: Language, Checker, and Math Library

- Lean 4 is simultaneously a functional programming language and a theorem prover.
- A *small kernel* checks proof terms; tactics help construct those terms.
- Mathlib organizes definitions, theorems, notation, tactics, and type classes into a library.

The right mindset for learning Lean: not proving the world from scratch, but *entering a reusable, checkable mathematical knowledge base*.

Mathlib: A Reusable Math Library

Mathlib contains:

- **Definitions** — types, structures, functions;
- **Theorems** — results with machine-checked proofs;
- **Notation** — mathematical symbols;
- **Type classes** — algebraic, order, topological structures.

Library-centered workflow: find existing definitions and theorems first; then write your own proof on top.

Theorem Names Are Searchable Addresses

Lean theorem names follow a pattern:

- **prefix** — Nat, Int, Set (namespace / domain);
- **core word** — add, mem, ext (operation or relation);
- **modifier** — comm, left (direction or property).

```
#check Nat.add_comm      -- Nat.add_comm : a + b = b + a
#check Int.add_comm
#check Set.mem_union
#check Finset.mem_union
```

Live Lean: [codeD1/D1.lean](#)

Different Namespaces, Same Short Name

Many mathematical objects have an “extensionality” theorem named `ext`; namespaces keep them distinct.

```
#check Set.ext
-- (h : forall x, x in s <-> x in t) : s = t

#check Finset.ext
-- (h : forall x, x in A <-> x in B) : A = B
```

This is why namespaces matter: short names can repeat; full names remain unique.

Live Lean: [codeD1/D1.lean](#)

File Organization Commands

```
import Mathlib                -- load a file or library
namespace Demo
  def double (n : Nat) : Nat := n + n
end Demo
section
  variable (n : Nat)
  example : Demo.double n = n + n := by rfl
end
open Nat
#check zero_add               -- same as Nat.zero_add
```

import: load a library; namespace: add a name prefix; section: scope local variables; open: drop a prefix temporarily.

Live Lean: `codeD1/D1.lean`

AI for Math: Where Does Lean Fit?

Explore AI and programs find patterns, constructions, or proof routes. (*Knuth's Cycles, IMO benchmarks*)

Translate Turn natural-language goals into formal statements. (*AlphaProof, formalization projects*)

Check The Lean kernel verifies the proof object. (*Lean formalization*)

This course teaches the *target language* that many AI provers must output and that formalization projects share.

Choosing AI for Lean Work

- Proof routes** Use a strong reasoning model. Ask for a list of lemmas and a proof outline; do not blindly trust a complete proof.
 - Retrieval** Use a tool with search and citations. Verify theorem names, versions, and dates; check in Mathlib or Lean.
 - Code repair** Use a coding or agent model. Supply the error message, goal state, and relevant imports; constrain the edit.
 - Local deploy** Use open-weight models for privacy or batch experiments.
- AI proposes candidates. Students understand and decompose. The Lean kernel gives the final verdict.*

What AI Is Good at in Math Learning

Exploration Quickly generate examples, conjectures, and proof routes; open up the search space.

Explanation Rephrase unfamiliar definitions, give analogies and small examples.

Collaboration Rewrite drafts, complete code, point out possible gaps.

Think of AI as a co-pilot: it broadens thinking, lowers the entry barrier, and speeds up drafts.

Hallucinations: The Dangerous Failure Mode

AI output that *looks like a proof* is not necessarily *a proof*:

- Fabricated theorem names or citations that do not exist.
- Skipped key lemmas; intuition passed off as conclusion.
- Symbols, conditions, or quantifiers quietly changed.
- Fluent-sounding reasoning that cannot be checked.

Why does this happen? Language models optimize for plausible text. They do not natively maintain a global dependency graph. Without external checking, errors can be very smooth. Treat AI output as a draft candidate, not as evidence.

Making AI Output Checkable

- 1 **Clarify the goal** — write a precise statement with all variables, conditions, and quantifiers.
- 2 **Decompose** — ask for a list of lemmas rather than one long proof.
- 3 **Check sources** — verify theorem names and API with `#check`, `exact?`, Loogle, or documentation.
- 4 **Machine check** — put key steps into Lean; let the kernel give the final judgment.

AI proposes. You understand and decompose. Lean checks.

Part II: Types and Functions

- Every expression in Lean has a type.
- Functions are maps between types.
- Lean infers types automatically when it can.

Start Writing Lean

```
import Mathlib

#check Nat          -- Nat : Type
#eval 2 + 3         -- 5

example (P : Prop) : P -> P := by
  intro h
  exact h
```

Core habit: after every line, read the Infoview panel to see the current type or goal.

Live Lean: `codeD1/D1.lean`

Three Core Ideas of Type Theory

Lean's proofs are not natural-language paragraphs; they are objects constructed inside type theory. Three intuitions:

- 1 **Every expression has a type.** `#check 2` gives `2 : Nat`.
- 2 **Functions are arrows between types.** `f : A → B` takes an `A` and returns a `B`. Lean checks that inputs and outputs match.
- 3 **Propositions are types.** `P : Prop` is a type. A proof of `P` is a *term of that type*.

Type Judgments: $a : A$

The basic sentence in Lean is a *type judgment*: $a : A$, read “a has type A.”

```
#check 42          -- 42 : Nat
#check Nat         -- Nat : Type
#check Type        -- Type : Type 1
#check 42 + 1      -- 42 + 1 : Nat
-- #check 42 + "hi" -- error: type mismatch
```

A type error does not mean False; it means the expression was rejected before truth or proof was even considered.

Live Lean: [codeD1/D1.lean](#)

Every Expression Has a Type

```
#check Nat          -- Nat : Type
#check 37           -- 37 : Nat
#check "hello"      -- "hello" : String
#check true         -- true : Bool
```

- #check asks for the type.
- #eval asks for the computed value.
- These are two different questions.

Live Lean: `codeD1/D1.lean`

#check, #reduce, #print

```
def onePlusOne : Nat := 1 + 1

#check onePlusOne    -- onePlusOne : Nat
#reduce onePlusOne    -- 2
#print onePlusOne     -- def onePlusOne : Nat := 1 + 1
```

#check What type does this name or expression have?

#reduce Reduce the expression to its normal form.

#print What is stored in the environment for this name?

Live Lean: `codeD1/D1.lean`

Types Also Have Types: Universes

```
#check Nat      -- Nat : Type
#check Prop     -- Prop : Type
#check Type     -- Type : Type 1
#check Type 1   -- Type 1 : Type 2
-- Sort 0 = Prop, Sort 1 = Type, Sort u : Type (u+1)
```

Why layering? If there were one “type of all types” $U : U$, self-reference would reintroduce paradoxes. Lean uses $\text{Type } u : \text{Type } (u+1)$ to avoid them.

Live Lean: [codeD1/D1.lean](#)

Why Not $U : U$? Russell's Problem

The naive idea: a “set of all sets that do not contain themselves.”

$$R = \{x \mid x \notin x\} \quad \text{Is } R \in R?$$

Both answers lead to contradiction. Lean avoids this with universe levels:

```
#check Type      -- Type : Type 1
#check Type 1    -- Type 1 : Type 2
-- Type u : Type (u + 1)
```

Live Lean: [codeD1/D1.lean](#)

Functions: Basic Building Blocks

```
def double (n : Nat) : Nat :=  
  n + n  
  
#check double      -- double : Nat -> Nat  
#eval double 21    -- 42
```

Parts of a definition:

- `def` — give the object a name;
- `(n : Nat)` — an explicit input parameter;
- `: Nat` — the return type;
- `n + n` — the function body.

Live Lean: [codeD1/D1.lean](#)

Anonymous Functions and Higher-Order Functions

```
#check fun x : Nat => x + 1

def applyTwice (f : Nat -> Nat) (x : Nat) : Nat :=
  f (f x)

#eval applyTwice (fun n => n + 1) 10  -- 12
```

A function can take another function as input. This is the core of functional programming.

Live Lean: [codeD1/D1.lean](#)

Multi-Argument Functions: Currying

```
def add3 (a b c : Nat) : Nat := a + b + c

#check add3          -- add3 : Nat -> Nat -> Nat -> Nat
#check add3 1        -- add3 1 : Nat -> Nat -> Nat
#check add3 1 2      -- add3 1 2 : Nat -> Nat
#check add3 1 2 3    -- add3 1 2 3 : Nat
```

`add3 1` is still a function waiting for two more arguments. Partial application comes for free in Lean.

Live Lean: [codeD1/D1.lean](#)

Implicit Arguments: Lean Infers Them

```
def myConst {A B : Type} (a : A) (_ : B) : A := a

#eval myConst 5 "ignored" -- 5
```

Curly-bracket parameters `{A B : Type}` are inferred by Lean from the surrounding context. When starting out, read implicit arguments as “type information that Lean fills in automatically.”

Live Lean: [codeD1/D1.lean](#)

Three Bracket Types

The bracket style on a parameter tells Lean *who provides it*.

`(x :  $\alpha$ )` **Explicit**. You supply it. Used for the main mathematical objects.

`{ $\alpha$  : Type}` **Implicit**. Lean infers it from later arguments or the goal.

`[Monoid  $\alpha$ ]` **Instance**. Lean searches the instance database (algebraic structure, order, topology, ...).

```
#check List.length      -- hides inferred arguments
#check @List.length     -- shows all arguments
#check @Nat.add_comm    -- shows [AddCommMonoid] instance
```

Live Lean: [codeD1/D1.lean](#)

Calling Functions with Implicit Arguments

```
def same {alpha : Type} (x : alpha) : alpha := x

#check same 3          -- same 3 : Nat  (alpha inferred)
#check @same Nat 3      -- explicit: alpha = Nat

example {alpha : Type} [AddMonoid alpha] (x : alpha) :
  x + 0 = x := by simp

example : same (alpha := Nat) 3 = 3 := by rfl
```

- @f shows all hidden arguments.
- f (alpha := Nat) supplies a named implicit argument.
- [AddMonoid alpha] triggers instance search.

Live Lean: [codeD1/D1.lean](#)

Part III: Curry-Howard and Prop

- Data has types. Propositions also have types.
- A proof of P is a *term of type* P .
- Lean tactics are constructing such terms.

Dependent Functions: Π , $\forall$, and fun

```
#check ((n : Nat) -> Fin (n + 1))  
-- the output type depends on the input n  
  
#check (forall n : Nat, n = n)  
-- universally quantified proposition  
  
example : (forall n : Nat, n = n) := by  
  intro n  
  rfl
```

Read $\forall n, P\ n$ as “a function type: give any n , return a proof of $P\ n$.” Then `intro` is simply the function-argument step.

Live Lean: [codeD1/D1.lean](#)

Propositions as Types

$2 + 2 = 4$ is a `Prop`

- Ordinary data has types: `42 : Nat`, `true : Bool`.
- Propositions are also types: `P : Prop`.
- A *proof* of `P` is a term of that type.

Reading `h : P` as “`h` has type `P`” and as “`h` is a proof of `P`” are *the same thing*.

The Curry-Howard Correspondence

Logic	Type theory
Proposition P	Type P
Proof of P	Term $p : P$
Proof of $P \rightarrow Q$	Function $f : P \rightarrow Q$
Use hypothesis $h : P$	Apply term h

Read the colon as “has type”: $h : P$ is both “ h has type P ” and “ h is a proof of P .”



- Swedish logician and philosopher of mathematics
- Developed **Martin-Löf Type Theory** (1972–1984) — the direct theoretical ancestor of Lean's type system
- Key insight: propositions-as-types is not a metaphor but a literal identity
- Introduced **dependent types**: types that depend on values (e.g. $\text{Vec } n$ for a vector of length n)
- His work made it possible to use type theory as both a *programming language* and a *proof system*

Logic	Proof construction	What to do
True	<code>True.intro</code>	trivial; one proof
False	no constructor	nothing proves it
$P \rightarrow Q$	<code>fun hP => hQ</code>	give P, return Q
$P \wedge Q$	<code>And.intro hP hQ</code>	supply both proofs
$P \vee Q$	<code>Or.inl hP</code> or <code>Or.inr hQ</code>	supply one side
$\text{not } P$	<code>fun h => absurd</code>	$P \rightarrow \text{False}$

Curry-Howard in Lean

```
example (P Q : Prop) : P -> Q -> And P Q := by
  intro hP
  intro hQ
  exact And.intro hP hQ

#check And.intro
-- And.intro : P -> Q -> P /\ Q
```

- `intro hP` puts the proof of `P` into context.
- `And.intro hP hQ` assembles both proofs into a proof of `And P Q`.

Tactics are not magic commands; they construct a term whose type is the current goal.

Live Lean: [codeD1/D1.lean](#)

- Tactics are a way to build a proof term step by step.
- Each tactic changes the goal shown in the Infoview panel.
- Read the goal before each tactic; confirm it changes as expected.

Reading the Lean Infoview

When your cursor is inside a `by` block, VS Code shows the **Lean Infoview** panel:

```
P Q : Prop           -- context (hypotheses)
hP : P
hPQ : P -> Q
----- Goal -----
Q                    -- what you need to prove
```

- Everything *above* the line is the **context**: names and their types that you can use.
- Everything *below* the line is the **goal**: what remains to be proved.
- Each tactic transforms the goal; move your cursor line by line to watch it change.

exact: Closing a Goal

```
example (P : Prop) (h : P) : P := by
  exact h
```

Infoview reading:

- Context: $h : P$
- Goal: P
- `exact h` closes the goal because h has exactly type P .

`exact` means: I already have a term whose type matches the goal.

Live Lean: `codeD1/D1.lean`

intro: Proving Implications

```
example (P : Prop) : P -> P := by
  intro h
  exact h
```

Goal changes:

- 1 Before `intro h`: goal is $P \rightarrow P$
- 2 After `intro h`: context gains $h : P$; goal is P
- 3 `exact h` closes it.

`intro` moves an assumption from the goal into the context.

Live Lean: [codeD1/D1.lean](#)

apply: Using a Theorem Backwards

```
example (P Q : Prop) (hPQ : P -> Q) (hP : P) : Q := by
  apply hPQ
  exact hP
```

- Goal before `apply hPQ`: Q
- Goal after: P
- `exact hP` closes it.

`apply` reduces the current goal to the premises of a known implication.

Live Lean: `codeD1/D1.lean`

constructor: Splitting a Conjunction

```
example (P Q : Prop) (hP : P) (hQ : Q) : And P Q := by
  constructor
  exact hP
  exact hQ
```

constructor on `And P Q` creates two subgoals:

- 1 After constructor: goal 1 is `P`, goal 2 is `Q`.
- 2 `exact hP` closes goal 1.
- 3 `exact hQ` closes goal 2.

Live Lean: [codeD1/D1.lean](#)

cases: Splitting a Disjunction

```
example (P Q : Prop) : Or P Q -> Or Q P := by
  intro h
  cases h with
  | inl hP => exact Or.inr hP
  | inr hQ => exact Or.inl hQ
```

cases h on Or P Q creates two branches:

- | inl hP => branch: context has hP : P
- | inr hQ => branch: context has hQ : Q

Each branch must close the goal independently.

Live Lean: [codeD1/D1.lean](#)

Negation, False, and contradiction

```
example (P : Prop) (hP : P) (hNotP : Not P) : False := by
  contradiction

example (P : Prop) (hP : P) (hNotP : P -> False) : False := by
  contradiction
```

- `Not P` is defined as `P -> False`.
- `False` has no constructor; nothing proves it.
- When context contains both `P` and `Not P`, `contradiction` finds the conflict automatically.

Live Lean: `codeD1/D1.lean`

use: Witness for Existence

```
example (m : Nat) : Exists (fun n : Nat => 0 + n = m) := by
  use m
  simp
```

- `use m` provides the concrete witness `m`.
- The remaining goal is the property: $0 + m = m$.
- `simp` closes it.

Exists is written $\exists$ in Lean: $\exists n, 0 + n = m$.

A proof of an existential = a concrete witness + a proof that it satisfies the property.

Live Lean: `codeD1/D1.lean`

Definitional Equality: What rfl Checks

```
def onePlusOne : Nat := 1 + 1

#reduce (fun n : Nat => n + 1) 3  -- 4
#reduce 2 + 3                    -- 5
#reduce onePlusOne               -- 2
```

Two expressions are *definitionally equal* if the Lean kernel reduces both to the same normal form. `rfl` closes a goal exactly when both sides reduce the same way.

Live Lean: [codeD1/D1.lean](#)

rfl: Same After Unfolding

```
example (a : Nat) : a + 0 = a := by
  rfl                                -- works: definitionally equal

example (a : Nat) : 0 + a = a := by
  exact Nat.zero_add a             -- needs a theorem
```

- $a + 0 = a$: Lean reduces both sides to the same thing.
- $0 + a = a$: reduction does not equate them; the theorem `Nat.zero_add` is needed.

Live Lean: `codeD1/D1.lean`

When rfl Fails

```
example : 2 + 3 = 5 := by rfl      -- succeeds

-- example : 2 + 2 = 5 := by rfl  -- fails: not equal

example (a : Nat) : 0 + a = a := by
  exact Nat.zero_add a
```

When rfl fails:

- Use #reduce to see what each side computes to.
- Search for a theorem using the goal's shape.
- Use rw or simp to reduce it to something rfl can handle.

Read rfl as “please compare by definition,” not “search for a proof.”

Live Lean: [codeD1/D1.lean](#)

rw: Rewriting with an Equation

```
example (a b c : Nat) (h : a = b) : a + c = b + c := by
  rw [h]           -- replaces a with b in the goal
```

```
example (a b c : Nat) (h : a = b) : c + b = c + a := by
  rw [h.symm]      -- replaces b with a in the goal
```

- `rw [h]` rewrites left-to-right using `h`.
- `rw [h.symm]` or equivalently `rw [← h]` rewrites right-to-left (`←` is the idiomatic Lean 4 form).
- `rw` gives precise control over exactly which equality is used.

Live Lean: `codeD1/D1.lean`

simp: Automatic Simplification

```
example (a : Nat) : a + 0 = a := by
  simp

def square (n : Nat) : Nat := n * n

example : square 3 = 9 := by
  simp [square]
```

- `simp` applies a library of standard simplification lemmas.
- `simp [f]` additionally unfolds the definition `f`.
- `simp` is automatic and fast; it may close goals that would need several `rw` steps.

Live Lean: [codeD1/D1.lean](#)

rw vs simp: When to Use Which

	<code>rw [h]</code>	<code>simp / simp [h]</code>
When	you know exactly which equation to use	standard cleanup or goal contains simp lemmas
Control	precise, one step	automatic
Speed	slower to write	fast
Use for	targeted rewrites	finishing off a goal

Two More Useful Tactics: omega and decide

```
-- omega: linear arithmetic over Nat and Int
example (n : Nat) (h : n > 3) : n >= 4 := by omega

example (a b : Nat) : a + b = b + a := by omega

-- decide: any decidable proposition with finite cases
example : 2 + 3 = 5 := by decide

example : (3 : Nat) < 7 := by decide
```

- omega solves linear arithmetic goals automatically — addition, subtraction, inequalities over Nat/Int.
- decide evaluates a decidable proposition at compile time; works whenever the kernel can reduce the goal to True.
- Use these before reaching for rw or simp on numeric goals.

The session-opening code snippet follows this route:

```
import Mathlib
#check Nat          -- step 1: see a type
#eval 2 + 3         -- step 1: compute a value

def double (n : Nat) : Nat := n + n -- step 2: write a function
#eval double 21

example (P : Prop) : P -> P := by    -- step 3: enter proof mode
  intro h
  exact h
```

First check types, then define functions, then prove.

Live Lean: `codeD1/D1.lean`

Nine Actions to Remember

Action	What it does
<code>#check e</code>	show the type of expression <code>e</code>
<code>#eval e</code>	compute the value of <code>e</code>
<code>intro h</code>	introduce a hypothesis or variable
<code>exact h</code>	close the goal with an exact term
<code>apply h</code>	reduce goal to the premises of <code>h</code>
<code>constructor / cases</code>	split a goal / split a hypothesis
<code>use t</code>	supply witness <code>t</code> for an existential goal
<code>rw [h] / simp</code>	rewrite / auto-simplify
<code>omega / decide</code>	linear arithmetic / evaluate decidable

Suggested Exercises

Work through these in `lean-code/D1.lean`. After each tactic line, record how the Infoview goal changes.

```
-- 1. implication chaining (intro, apply, exact)
example (P Q R : Prop) : (P -> Q) -> (Q -> R) -> P -> R := by
  sorry

-- 2. rewriting (rw, omega)
example (a b : Nat) (h : a = b) : a + a = b + b := by
  sorry

-- 3. And commutativity (intro, cases, constructor, exact)
example (P Q : Prop) : P /\ Q -> Q /\ P := by
  sorry

-- 4. existential witness (use, omega)
example : Exists (fun n : Nat => n > 10) := by
  sorry
```

Learning Lean is not memorizing commands; it is learning to *read the goal*, *choose an action*, and *confirm the goal changes*.